

MediCore Hospital System

Design Justification Document

Course	OOP CSE142 — Homework 2
Name	Kartar Singh
ERP ID	32431
Instructor	Sir Behraj Khan

This document covers the full design of the MediCore C++ hospital system built for HW2. It explains the class hierarchy, design decisions made, how memory is managed, and what each test case is checking.

Section	Topic
1	Class Hierarchy — Inheritance trees and how classes relate
2	Design Decisions — Reasons behind key choices in the design
3	OOP Concepts — How inheritance, polymorphism, encapsulation are used
4	Operator Overloading — Which operators are overloaded and why
5	Memory Management — Ownership model, copy/move semantics
6	Test Cases — TC-01 through TC-10

1. Class Hierarchy

The system uses two main inheritance trees plus a few standalone classes. Inheritance is used to avoid repeating shared fields and to allow polymorphic behaviour where different types need to do the same thing differently.

Person Tree

```
Person (abstract)
/ \
Patient StaffMember (abstract)
/ | \
GP Surgeon Nurse
```

Class	Inherits from	Abstract?	What it adds
Person	—	Yes	name, dob, id, phone — pure display()
Patient	Person	No	diagnosis, ward, treatments, bill generation
StaffMember	Person	Yes	salary, department — pure calculateBillingRate()
GP	StaffMember	No	consultationFee — billing rate is this fee
Surgeon	StaffMember	No	specialisation, operationFee
Nurse	StaffMember	No	assignedWard, careHourlyFee

Ward Tree

```
Ward (abstract)
/ | \
GeneralWard ICU SurgicalWard
```

Class	Daily Rate	Admission Rule
Ward	Abstract	canAdmit() is pure virtual
GeneralWard	PKR 500	Always returns true — accepts any patient
ICU	PKR 2000	Only if patient.isCritical() == true
SurgicalWard	PKR 1500	Only if patient has a scheduled operation

Standalone Classes

Class	Role
Treatment	Value object — stores treatment name, cost, and staff who performed it
Bill	Value object — treatment total + ward stay fee + fixed PKR 500 admin fee
Appointment	Value object — stores patient, staff, date, and time slot
AppointmentBook	Holds all appointments; rejects any add that would double-book a staff slot
Hospital	The main system — owns all patients, manages admit/discharge/queries

2. Design Decisions

Some choices in the design are not immediately obvious. This section explains the reasoning behind each one.

Person as base class

All people in the system share common data like name, ID, and phone number. These are placed in a single base class to avoid repeating the same fields in Patient and StaffMember. `display()` is pure virtual because there is no meaningful generic summary — each type must define its own.

StaffMember as a separate abstract layer

Doctors and nurses share salary and department but their billing methods are completely different. A separate StaffMember layer sits between Person and the concrete staff types. `calculateBillingRate()` is pure virtual so calling code never needs to know the concrete type.

Ward design with virtual `admit()`

Each ward type has a different rule for accepting patients. Instead of checking the ward type in main with if/else, `admit()` is virtual and each subclass enforces its own rule. This keeps main clean and follows the open-closed principle.

Patient owns its Treatment array

Each patient stores treatments as a dynamically-grown raw pointer array (no smart pointers as per constraints). A proper deep copy constructor ensures that copying a patient gives a fully independent copy of all its treatments. This is what TC-06 checks.

Hospital is the sole owner of patients

Hospital allocates Patient objects and is the only class that frees them. Ward only holds non-owning pointers. This avoids double-free when a patient is discharged — removing the patient from the Ward just erases the pointer, it does not free anything.

Bill as a simple value type

Bill holds three numbers and supports operator+, ==, and <<. It has no heap allocation so the compiler-generated copy works correctly. The PKR 500 admin fee is factored into each individual bill. operator+ combines the sub-totals so that `combined.total() == b1.total() + b2.total()` holds correctly — this is what TC-02 verifies.

AppointmentBook checks for double-booking before inserting

Before adding a new appointment the book scans existing ones for the same staff member, date, and time slot. If a conflict is found it returns false without inserting. This prevents a staff member from being booked twice at the same time.

Lambda-based filter and sort

`filterPatients` and `sortPatients` accept `std::function` so the caller can pass any lambda. No hard-coded conditions exist inside those functions. This satisfies TC-08 and makes the query interface flexible without needing extra subclasses.

3. OOP Concepts Applied

Concept	Where it appears	How
Encapsulation	All classes	Every field is private or protected. External code uses methods only.
Inheritance	Person tree, Ward tree	Shared fields declared once in base class, never duplicated in subclasses.
Polymorphism	display(), admit(), calculateBillingRate()	Virtual functions — calling code uses a base pointer and the right function.
Abstraction	Person, StaffMember, Ward	Cannot be instantiated directly. They define contracts that concrete types implement.
Composition	Patient owns Treatment[] Hospital owns Patient*	Complex objects are built from simpler ones. Ownership is explicit.

4. Operator Overloading

Operators are only overloaded where the meaning transfers naturally into the domain.

Operator	Class	What it does
operator+	Bill	Combines two bills — adds treatment totals and ward fees into a new Bill
operator==	Bill	True if total() values match — value equality, not object identity
operator<<	Bill	Streams an itemised breakdown — friend function to access private fields
operator<	Ward	True if this ward has lower occupancy percentage than the other
operator>	Ward	True if this ward has higher occupancy percentage
operator==	Ward	True if occupancy percentages are equal

Note on Bill operator+: When combined = b1 + b2 is called, combined.total() == b1.total() + b2.total() holds correctly — this is what TC-02 verifies. The combined bill's total is the arithmetic sum of both individual totals.

Note on Ward comparison: Comparison is by occupancy percentage, not raw patient count. A ward with 7 of 8 beds filled (87.5%) is correctly greater than one with 5 of 20 (25%), even though 5 < 7 as raw numbers. The ratio is the meaningful metric here.

5. Memory Management

Ownership Model

Hospital is the sole owner of every heap object. When Hospital's destructor runs it deletes every Patient* in both live and archive lists. Ward holds non-owning pointers to patients — removing a patient from a ward just erases the pointer, it does not free anything.

Class	What it allocates	Who frees it
Patient	Treatment** (dynamically grown array)	Patient destructor — frees each Treatment then the array
Hospital	Patient* (live and archive lists)	Hospital destructor — frees every patient it holds
Ward	Patient* array (non-owning pointers)	Ward destructor frees the array only, NOT the Patient objects
AppointmentBook	Appointment* objects	AppointmentBook destructor
Bill / Treatment / Appointment	Nothing — value types with no heap	N/A — compiler-generated destructor is sufficient

Important: Only one class is responsible for deleting each object. This avoids double-free errors and dangling pointers. Ward is intentionally non-owning — if it freed patients, Hospital would still hold a pointer to freed memory.

Patient — Rule of Five

Patient needs explicit copy and move operations because Hospital::discharge() move-constructs the patient into the archive, and TC-06 requires the copy constructor to be a true deep copy.

Special member	What it does
Copy constructor	Deep copies every field — the treatments array gets its own independent copy
Copy assignment	Self-assignment check first, then deep copies all fields
Move constructor	Transfers the treatments pointer and nulls the source pointer to prevent double-free
Move assignment	Same pattern — transfer then reset source
Destructor	Frees every Treatment* in the array, then frees the array itself

6. Test Cases

Each test case is a standalone function in main.cpp. They use a check() helper that prints PASS or FAIL to stdout.

TC-01 | Polymorphic Display

Setup: One GP, one Surgeon, one Nurse, and one Patient stored in a vector. display() is called through the base pointer on each.

- ✓ GP output includes consultation fee
- ✓ Surgeon output includes specialisation and operation fee
- ✓ Nurse output includes assigned ward and care fee
- ✓ Patient output includes diagnosis and admission date

Justification: Confirms virtual dispatch routes display() to the correct subclass. No if/else or type-checking is used in the calling code.

TC-02 | Billing Operator Overloading

Setup: Two patients are given treatments and bills are generated.

- ✓ combined.total() == b1.total() + b2.total() (operator+)
- ✓ b1 != b2 (operator==)
- ✓ A copy of b1 equals b1 (value equality)

Justification: Checks all three overloaded Bill operators. The combined bill check verifies the arithmetic identity holds.

TC-03 | Ward Comparison Operators

Setup: Ward A has 20 beds with 5 patients (25%). ICU has 8 beds with 7 patients (87.5%).

- ✓ w1 < w2 (25% < 87.5%)
- ✓ w2 > w1
- ✓ !(w1 == w2)

Justification: Validates that Ward comparison uses occupancy ratio, not raw patient count.

TC-04 | ICU Admission Rule Enforcement

Setup: A critical patient and a non-critical patient are admitted to ICU and GeneralWard.

- ✓ ICU accepts critical patient
- ✓ ICU rejects non-critical patient
- ✓ GeneralWard accepts any patient

Justification: Directly tests the polymorphic admit() implementation. No ward-type check in main.

TC-05 | Double-Booking Prevention

Setup: Three appointments added: first valid, second conflicts (same staff/date/slot), third valid (different slot).

- ✓ First add returns true

- ✓ Conflicting add returns false
- ✓ Different slot add returns true

Justification: Tests AppointmentBook conflict detection. Conflict is staff + date + slot based.

TC-06 | Copy Independence

Setup: A patient with one treatment is copy-constructed. A second treatment is added to the copy only.

- ✓ Original still has exactly 1 treatment
- ✓ Copy has 2 treatments

Justification: Verifies the copy constructor performs a proper deep copy of the treatments array.

TC-07 | Move Semantics — Discharge and Archive

Setup: A patient is admitted then discharged via Hospital::discharge().

- ✓ isAdmitted() returns false after discharge
- ✓ isArchived() returns true after discharge
- ✓ No crash on scope exit — move left source in valid state

Justification: Tests the full admit-discharge flow and exercises move semantics on Patient.

TC-08 | Composable Filtering with Lambdas

Setup: Six patients admitted (3 critical in ICU, 3 non-critical in Ward A). filterPatients and sortPatients called with lambdas.

- ✓ Filter returns exactly 3 critical ICU patients
- ✓ Sorted list is in non-decreasing bill order

Justification: Shows filterPatients and sortPatients work with arbitrary lambdas. No hard-coded conditions exist inside those functions.

TC-09 | Staff Billing Rate Polymorphism

Setup: A GP, Surgeon, and Nurse stored in a vector. calculateBillingRate() called through the base pointer.

- ✓ GP rate is correct
- ✓ Surgeon rate is correct
- ✓ Nurse rate is correct
- ✓ All three rates are different from each other

Justification: Mirrors TC-01 for calculateBillingRate(). No typeid or dynamic_cast used.

TC-10 | Revenue Report

Setup: Three patients admitted to Ward A, each with treatments. All discharged. wardRevenue() called.

- ✓ wardRevenue() returns the correct sum of all three bills

Justification: Tests that revenue is calculated from archived patients only and that generateBill() produces the correct total.
